



UNIVERSIDADE FEDERAL DO MARANHÃO
BACHARELADO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA
CAMPUS SÃO LUÍS

Millena Gomes Andrade de Menezes Braga
Cássio Herberth Rodrigues Araújo
Arthur Felipe Mourão de Oliveira
Alana Mayara Silva Monteiro

Relatório Técnico: Otimização Arquitetural e Consolidação do Projeto Jogo da Cobrinha (Snake) em Assembly MIPS

SÃO LUÍS
2026



**Millena Gomes Andrade de Menezes Braga
Cássio Herberth Rodrigues Araújo
Arthur Felipe Mourão de Oliveira
Alana Mayara Silva Monteiro**

Relatório Técnico: Otimização Arquitetural e Consolidação do Projeto Jogo da Cobrinha (Snake) em Assembly MIPS

Relatório referente a experimentos realizados na cadeira de Arquitetura de Computadores do curso de Ciência e Tecnologia da Universidade Federal do Maranhão (UFMA) para obtenção da nota.

**SÃO LUÍS
2026**

1. Resumo Executivo.....	4
2. A Gênese do Projeto: O Protótipo Inicial.....	4
3. Evolução Arquitetural e Resolução de Problemas (Troubleshooting).....	5
Problema 1: Sobrecarga na ULA e Cálculo de Pixels.....	5
Problema 2: Rigidez Sintática e Alocação de Memória no MARS.....	5
Problema 3: Conflito de I/O no Display de 7 Segmentos.....	5
Problema 4: A Barreira do Sandbox e o Middleware Web.....	6
4. Arquitetura Definitiva e Fluxo de Dados (Diagrama).....	7
5. O Dashboard Web e Sistema Inteligente de Ranking.....	8
7. Conclusão.....	10
8. Referências Bibliográficas.....	10

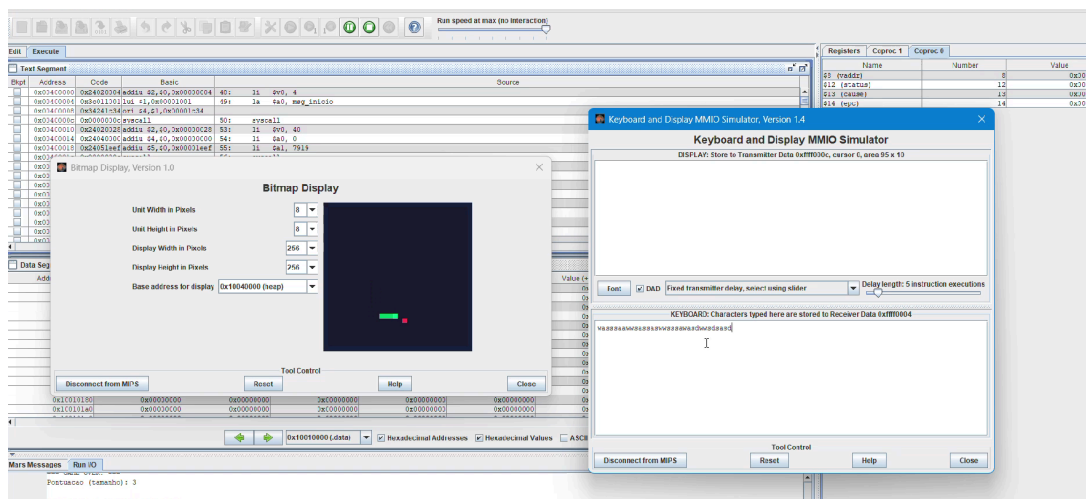
1. Resumo Executivo

Este documento formaliza a conclusão do projeto de desenvolvimento do jogo "Snake", programado integralmente em linguagem de baixo nível (Assembly MIPS) utilizando o simulador MARS. O projeto evoluiu de um protótipo básico de movimentação em matriz para um estudo de caso complexo de Arquitetura de Sistemas. A versão final engloba otimização de instruções na ULA, manipulação de periféricos via MMIO (Memory-Mapped I/O) e a construção de um middleware que integra o processador simulado a um dashboard web responsivo de high scores.

2. A Gênese do Projeto: O Protótipo Inicial

No início do ciclo de desenvolvimento, o escopo do projeto era focado estritamente na validação da lógica do jogo em baixo nível. O protótipo original (versão 1.0) possuía características limitadas:

- **Motor Gráfico Básico:** A renderização ocorria em uma malha menor, e o cálculo de posição dos pixels (X, Y) na memória de vídeo utilizava a instrução de multiplicação padrão (mult), que consumia múltiplos ciclos de clock do processador.
- **Memória Estática Restrita:** O corpo da cobra era gerenciado por vetores pequenos preenchidos manualmente com valores inteiros, limitando o crescimento do jogador e a duração da partida.
- **Sistema Isolado:** Não havia sistema de pontuação visível em hardware externo (display) nem qualquer conexão com ambientes fora do simulador MARS. A pontuação existia apenas nos registradores internos.



3. Evolução Arquitetural e Resolução de Problemas (Troubleshooting)

À medida que o escopo foi ampliado para incluir telas maiores (64×64), integração de hardware (MMIO) e comunicação web, a equipe enfrentou e solucionou diversos gargalos de engenharia de software e hardware.

Problema 1: Sobrecarga na ULA e Cálculo de Pixels

O Desafio: Com o aumento da tela para 64×64, a instrução `mult` no cálculo da coordenada de vídeo ($\text{Endereço} = \text{Base} + (Y \times 64 + X) \times 4$) começou a representar um custo computacional alto.

A Solução: A equipe substituiu a multiplicação por um Deslocamento Lógico à Esquerda (`sl`). Deslocar os bits de `Y` em 6 posições equivale a multiplicar por 64, mas a operação é resolvida na ULA em apenas 1 ciclo de clock.

Problema 2: Rigidez Sintática e Alocação de Memória no MARS

O Desafio: Durante a refatoração para comprimir o código da leitura do teclado (WASD), o agrupamento de instruções separadas por ponto e vírgula (ex: `li $t4, 1; sw $t4, direcao;`) gerou erros críticos de compilação (`Invalid language element`). Além disso, a alocação de memória do corpo da cobra usando multiplicadores (`0:1020`) gerou falhas de leitura no segmento `.data`.

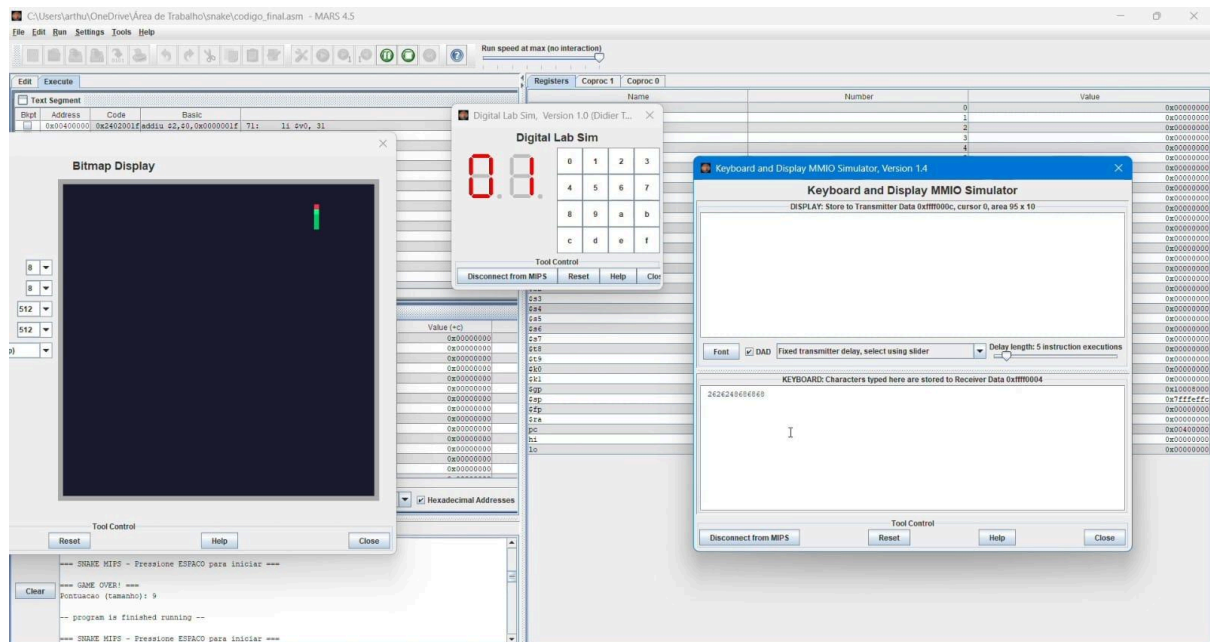
A Solução: O código foi padronizado seguindo a arquitetura RISC estrita (uma instrução por linha). Na memória, a sintaxe foi substituída pela diretiva `.space 16384` (alocando 4096 palavras brutas em bytes), garantindo estabilidade e evitando *stack overflow*.

Problema 3: Conflito de I/O no Display de 7 Segmentos

O Desafio: O hardware físico (display MMIO) estava registrando pontos indevidamente. O simples ato de pressionar a tecla "ESPAÇO" para sair da tela de espera inicial somava "1" ao placar, pois o bloco lógico de atualização da pontuação

estava solto no fluxo principal de execução da .text.

A Solução: A rotina matemática de divisão (dezenas e unidades) e conversão hexadecimal para os LEDs (0x5B, 0x6D, etc.) foi encapsulada na função isolada atualizar_display. Ela passou a ser invocada exclusivamente no *setup* e após a função de colisão com a maçã.



Problema 4: A Barreira do Sandbox e o Middleware Web

O maior desafio técnico da equipe foi exportar a pontuação do MIPS local para um *front-end* moderno.

O Desafio: Inicialmente, o simulador salvava o arquivo `score.bin` em diretórios ocultos do sistema operacional, tornando-o invisível para a página web. Além disso, quando o arquivo era encontrado, o servidor local (*Live Server*) forçava a atualização inteira da página (*refresh*), apagando o estado do jogo e quebrando a automação.

A Solução:

- A syscall 13 foi reprogramada para utilizar caminhos absolutos (ex: `C:/Users/alana/Desktop/...`), forçando o MARS a gravar dentro da pasta correta.

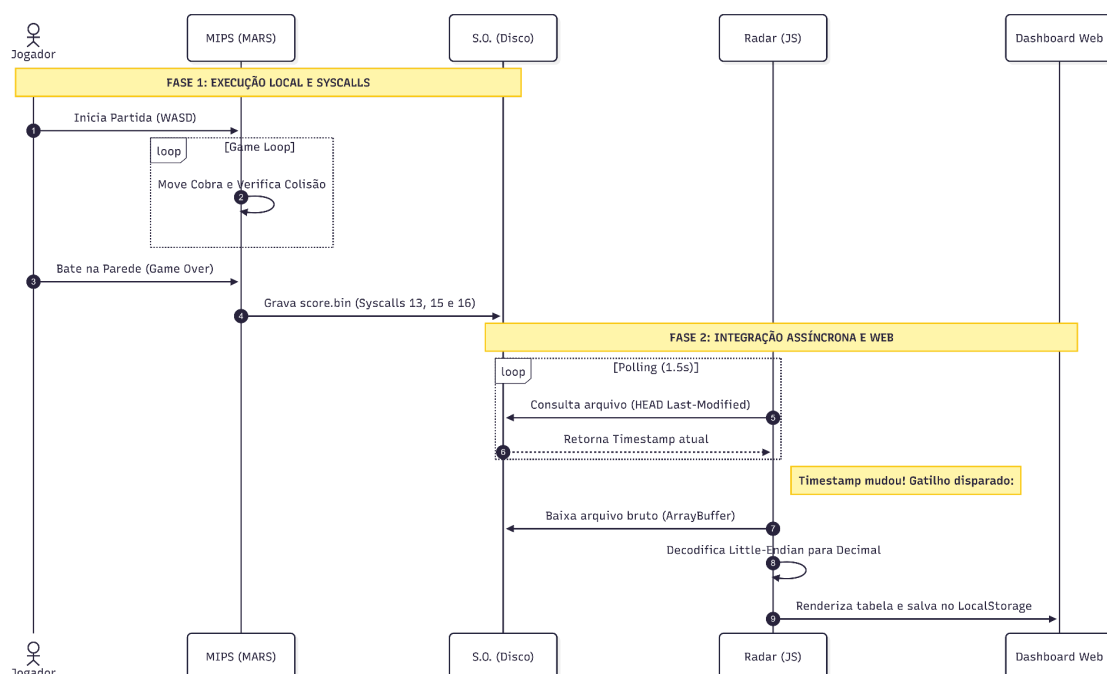
- Foi criado um diretório `.vscode` com configurações de exceção (`ignoreFiles: "**/*.*.bin"`) para estabilizar o servidor.

Problema 5: Conflito de Rotas no Windows e Caracteres de Escape

- O Desafio:** Ao migrar o projeto para a máquina oficial de testes, o arquivo `score.bin` parou de ser gerado na pasta correta. O problema ocorreu porque o caminho absoluto copiado do Windows possuía barras invertidas (`\`) e passava por pastas da nuvem (OneDrive) com caracteres especiais (ex: "Área de Trabalho"). O interpretador MIPS identificava o `\` como caractere de escape, corrompendo a leitura do diretório e fundindo os nomes das pastas em um único arquivo inválido.
- A Solução:** A equipe isolou o diretório do projeto transferindo-o diretamente para a raiz do disco rígido local (`C:/snake/`). No código-fonte, a sintaxe foi reescrita utilizando estritamente barras normais (`/`), garantindo que a `Syscall` de gravação operasse sem bloqueios de permissão (UAC) ou corrupção de *strings*.

4. Arquitetura Definitiva e Fluxo de Dados (Diagrama)

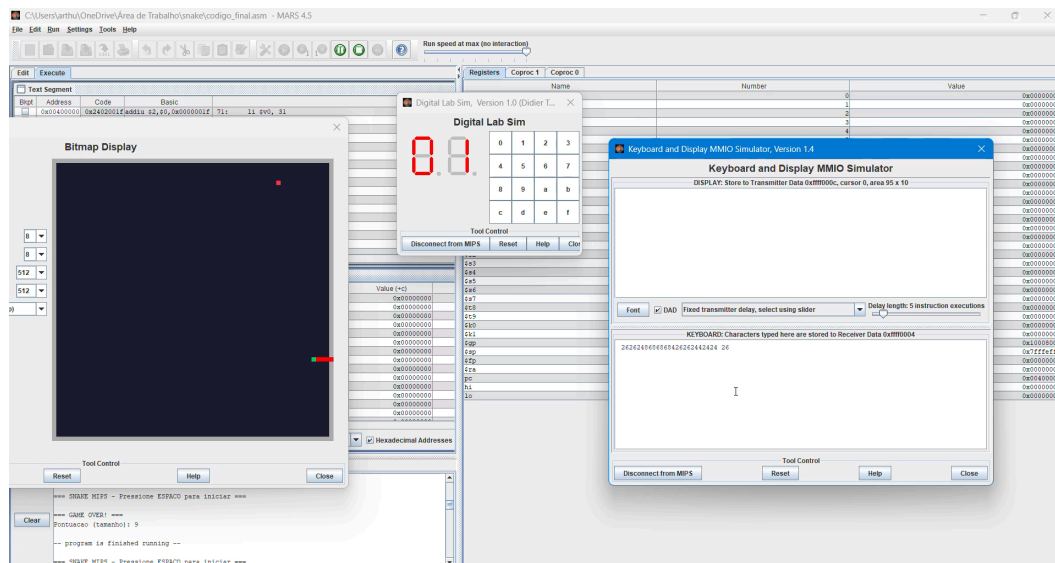
A solução final culminou em um ecossistema *cross-layer*, onde o baixo nível (processador/assembly) se comunica de forma assíncrona com o alto nível (navegador/JavaScript).

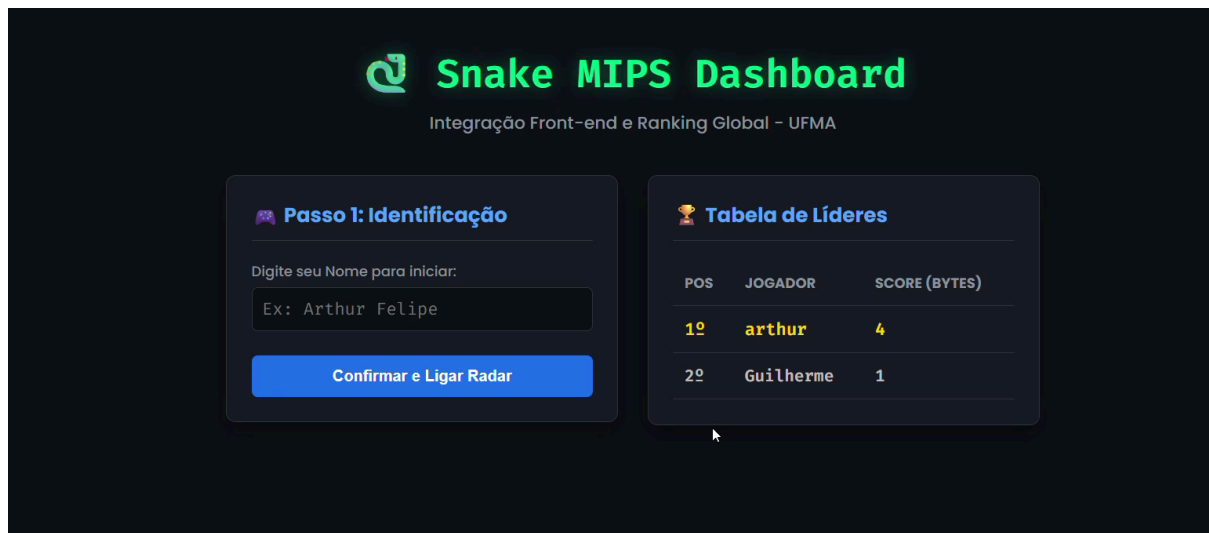


5. O Dashboard Web e Sistema Inteligente de Ranking

A interface *front-end* não apenas recebe os dados, mas atua como um sistema independente de validação:

- **Radar (Polling Assíncrono):** Utiliza requisições HEAD contínuas para não sobrecarregar a rede, baixando o binário bruto apenas quando a data de modificação física do arquivo muda.
- **Tradução Arquitetural:** O algoritmo decodifica os 4 bytes recebidos respeitando a arquitetura *little-endian* através do objeto `DataView(buffer).getInt32()`.
- **Banco de Dados (High Score):** O sistema varre o `localStorage` validando o nome do jogador (*case-insensitive*). A tabela de líderes só é atualizada se a nova pontuação ultrapassar o recorde anterior daquele mesmo usuário, evitando *spam* no banco de dados.





6. Cronograma de Execução e Fases do Projeto

O desenvolvimento do sistema foi estruturado em quatro fases incrementais, garantindo que a estabilidade do hardware fosse completamente validada antes da introdução da camada Web. O projeto foi concluído dentro do prazo estipulado para o encerramento do semestre letivo, conforme o detalhamento abaixo:

- Fase 1: Lógica Base e Protótipo (Abril/2026)

Estruturação dos vetores dinâmicos para o corpo da cobra e implementação da lógica de deslocamento bidimensional utilizando matrizes na seção .data.

- Fase 2: Interação de Hardware e Otimização (Maio/2026)

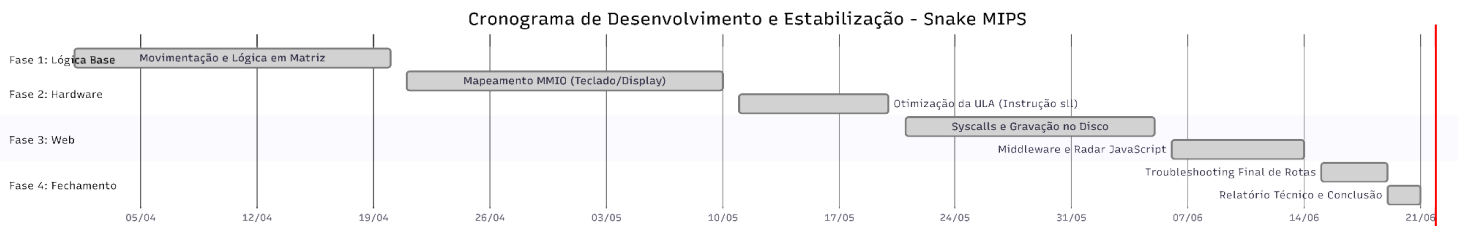
Mapeamento dos periféricos Memory-Mapped I/O (MMIO), habilitando o controle via teclado e saída visual no Bitmap Display e LEDs de 7 Segmentos. Implementação da otimização de processamento com deslocamento de bits (sll) na ULA.

- Fase 3: Construção do Middleware (Junho/2026)

Programação das Syscalls de gravação física no disco e desenvolvimento do Radar Assíncrono em JavaScript para capturar as pontuações em tempo real.

- Fase 4: Troubleshooting e Homologação (Junho/2026)

Resolução do gargalo de caminhos absolutos e caracteres de escape no sistema operacional. Finalização do Termo de Encerramento do Projeto (TEP).



7. Conclusão

O projeto Jogo da Cobrinha superou amplamente as exigências básicas do roteiro acadêmico. A equipe demonstrou capacidade de gerenciar falhas (*troubleshooting*), refatorar código de máquina, otimizar ciclos de clock e, o mais importante, conectar uma arquitetura histórica (MIPS) aos parademas modernos de desenvolvimento de software (sistemas distribuídos e web).

O sistema encontra-se validado e 100% estabilizado.

Situação do Projeto: Aprovado e Encerrado.

8. Referências Bibliográficas

- PATTERSON, David A.; HENNESSY, John L. **Computer Organization and Design: The Hardware/Software Interface**. 5ª ed. Morgan Kaufmann, 2013.
- VOLLMAR, K.; SANDERSON, P. **MARS: MIPS Assembler and Runtime Simulator**. Documentação Oficial e Guia de Utilização. Disponível em: [\[http://courses.missouristate.edu/KenVollmar/MARS/\]\(http://courses.missouristate.edu/KenVollmar/MARS/\)](http://courses.missouristate.edu/KenVollmar/MARS/).
- MDN WEB DOCS. **DataView - JavaScript**. Mozilla Foundation. Disponível em: [\[https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/DataView\]\(https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/DataView\)](https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/DataView).
- MDN WEB DOCS. **Fetch API**. Mozilla Foundation. Disponível em: [\[https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API\]\(https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API\)](https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API).

9. Repositório e Demonstração

- **Repositório do Código-Fonte (GitHub):**
<https://github.com/arthurmmourao/snake-assembly-mips/>
O repositório contém a totalidade do código-fonte desenvolvido em Assembly MIPS (`codigo_final.asm`), a infraestrutura de front-end (`index.html`) e os ficheiros de configuração do ambiente de desenvolvimento no VS Code.
- **Vídeo de Demonstração Prática (Google Drive):**
https://drive.google.com/file/d/1vs9DfqczGPiMnGjPomc1mx6rPHzuoAas/view?usp=drive_link
O vídeo apresenta a execução do jogo no simulador MARS, a interação em tempo real com os periféricos MMIO (teclado e displays) e o funcionamento assíncrono do dashboard web a receber as pontuações após o *Game Over*.